

Software Engineering II
Spring 2026

Bug Tracker System Report



Group 8:

Mengjia Yu
Roxanne Caicedo
Jorge Cuautle
Taiwo Ojo

Table of Contents

Functional Requirements	3
User View	5
Admin View	6
Staff View	7
Non-Functional Requirements	8
Interface Requirements	9
Testing	10
Roles of Team Members	11
Outcomes of the Project	12
Limitations	12
Conclusion	12

Functional Requirements

Our bug tracker system is designed as a web-based application that supports three main user roles: Admin, Staff, and User. The system includes several functional requirements that allow Admin, Staff, and User roles to manage bugs throughout the system. Each role has its own set of features and access controls, allowing all users to interact with only the parts of the system that are relevant to them. The system begins with a login module where all users enter their credentials and are directed to their role-based dashboard after successful authentication. Each dashboard is customized to display the homepage, menu, and features that are necessary for that specific role.

When a User logs in, the first page that loads is the *My Bugs* page, which displays any bug they have submitted to the system. Users are able to report a bug through the *Submit Bug* form. When reporting a bug, Users are required to provide details such as the *Bug Name*, *Source Application*, *Severity*, and *Impact*. After the bug report is submitted, the system generates a ticket number for the bug and sets its status field to *New*. The ticket number is used throughout the system to identify the bug, track its progress, and connect the actions of all roles involved in the bug resolution process. Users can also view and monitor the status of their submitted bugs. Once their bug is resolved, they can rate the solution with a 1–5 star rating. This allows the user to give feedback on how effective they found the bug solution. These ratings are visible to all users and roles. Finally, users can search all bugs in the system, view their details, solution, and rating, and comment on them.

Admins are responsible for managing the overall system. Their dashboard menu is divided into two main sections, *Bug Management* and *Staff Management*. When an Admin logs in, the first page that loads is the *New Bugs* page. This page displays all newly submitted bugs in the system that have not yet been assigned to a staff member. Admins *assign* new bugs to staff members, which updates the bug status to *Assigned*. They also *Approve* or *Reject* bug solutions submitted by staff members which updates the bug status to *Resolved* and *Assigned* respectively. In addition, Admins have the ability to edit a bug and update all of its details, such as *Severity*,

Impact, and *Staff Assignment*. Admins can also search all bugs in the system, view their details, solution, and rating, and comment on them. In regard to Staff Management, Admins can search all staff members and update their basic information, such as name and email. They can also add new staff members to the system, which creates a staff account directly in the database.

Staff focus on resolving bugs and providing bug solutions. When a Staff member logs in, their homepage displays all the bugs that have been assigned to them. These bugs are divided into three categories: *New Assignments*, *In Progress*, and *Resolved*. New Assignments are bugs that have recently been assigned to the Staff member and still need to be accepted. Once the Staff member accepts a bug, its status is updated to *In Progress* and the bug automatically moves to that section. After working on the bug, the Staff member can submit a solution and send it to the Admin for approval. At this point, the bug status is updated to *Pending Approval* until the Admin approves or rejects the solution. Staff can also search all bugs in the system, view their details, solution, and rating, and comment on them.

As described in each role's section, User, Admin, and Staff have access to the *Bug Search Module*. This allows all three roles to search through bugs by keyword, ticket number, or other bug details. They also have the option to view all bugs at once. Together, these features improve efficiency by making it easier to locate specific bug reports, especially when the system contains a large number of records. In addition, the Comment Module allows communication between roles by letting Admins, Staff, and Users add comments to bugs. The ticket number and status field are the main mechanisms that control how bug information flows through the system, from initial submission to staff assignment, solution approval, and final resolution. The Comment Module and Rating Module support this process by connecting the roles through communication and feedback. Comments allow Users, Staff, and Admins to discuss the bug and its solution, while ratings allow users to provide feedback on resolved bug solutions for future reference.

User view



User Dashboard — Welcome test user Logout

Main Menu [My Bugs](#) [Submit Bug](#) [Search All Bugs](#)

My Bugs

new bug
Ticket #: TK-1002
Severity: high
Impact: impact
Status: new
Source: app
[Comment](#)

some bug name
Ticket #: TK-1001
Severity: high
Impact: some impact
Status: resolved
Source: some application
[Comment](#) ★★★★★ (5/5)

Submit Bug

Bug Name

Source Application

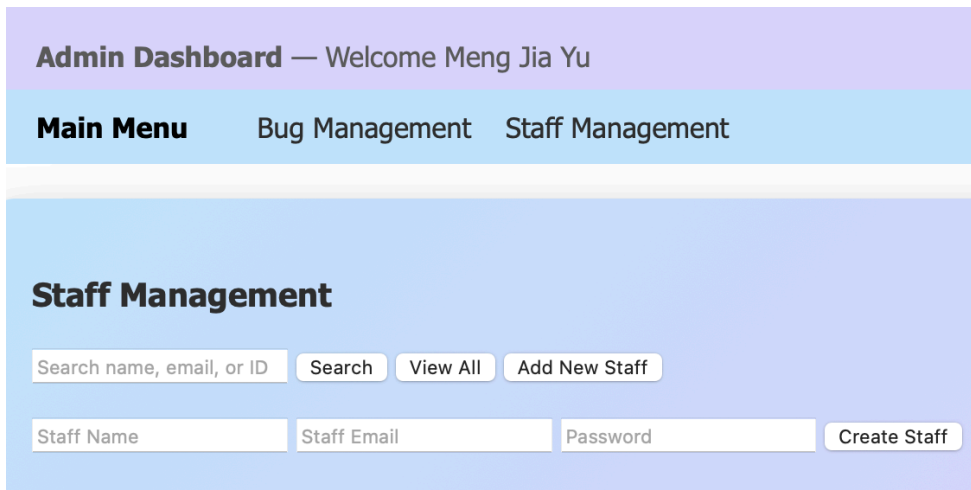
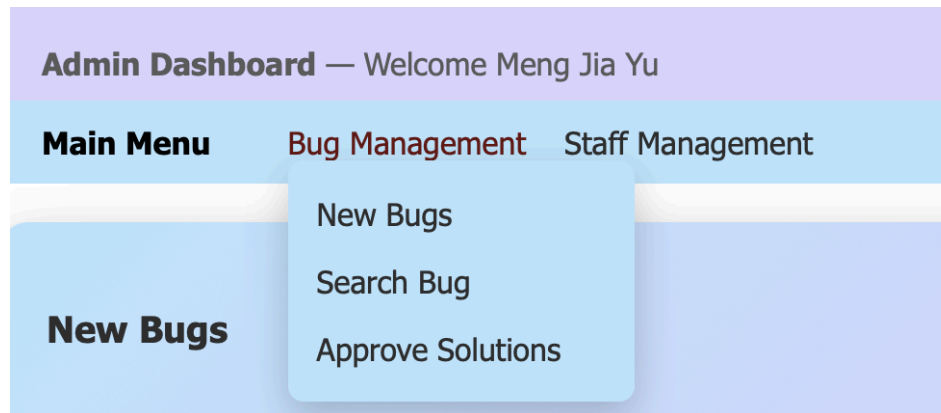
Severity

Select Severity 

Impact

[Submit](#)

Admin view



Assign / Edit Bug

Ticket Number:	TK-1001
Bug Name:	Login Crash
Severity:	High
Impact:	Users cannot access system
Status:	New
Source Application:	Mobile App
Assign to Staff:	Jackson Darrel (ID: 2)

Staff view



Staff Dashboard — Welcome James SmithLogout

Main MenuMy BugsSearch Bugs

New Assignments

No new assignments.

In Progress

Resolved Bugs

Staff Dashboard — Welcome James SmithLogout

Main MenuMy BugsSearch Bugs

Search Bugs

SearchView All

new bug

Ticket #: TK-1002
Severity: high
Impact: impact
Status: new
Source: app
User ID: 11
Staff ID:
Solution:

[Comment](#)

some bug name

Ticket #: TK-1001
Severity: high
Impact: some impact
Status: resolved

[Comment](#)

Non-Functional Requirements

In addition to the main functional requirements, our bug tracker system also addressed several non-functional requirements. Non-functional requirements are system characteristics such as reliability, security, usability, and performance. For our system, the two strongest non-functional requirements we implemented were usability and security.

Usability was our major focus because the system is designed for three different roles, Admin, Staff, and User. Each role has a customized dashboard that only displays the features needed for that role. This makes the system easy and intuitive to navigate and prevents users from being overwhelmed by options that do not apply to them or are not accessible to them. The interface also uses a consistent layout, menu structure, and content design throughout, which helps users move through pages and provides a cohesive user experience. Features like the Bug Search Module, the view all bugs option, status labels, and organized staff bug sections significantly improve usability by making bug information easy to find and quick to understand.

Security was also an important non-functional requirement we focused on. Our bug tracker system requires all Admin, Staff, and Users to log-in before accessing their dashboard. Passwords are stored in a hashed format in the database, which helps protect account information and keep it confidential. In addition, the system used role-based access control, where each page checks the user's session and role before allowing access. This prevents one role from accessing pages or actions that belong to another. The system also uses prepared statements for database queries, which helps protect it from SQL injection attacks.

Finally, reliability and performance were also considered in the system design. The system supports consistent data storage by saving each bug in the database with a unique ticket number and a dynamic status field, which helps keep bug data organized as it moves through the resolution process. Since status is essential to this data flow, only Admins have the ability to directly change a bug's status. Otherwise, the status is dynamically updated through key actions, such as assigning a bug, accepting it, submitting a solution, or approving a solution. Comments, ratings, staff assignments, and solutions are also saved in the database so that the system can maintain a consistent record of each bug. Ratings do not have a delete option, allowing them to serve as a reliable source of feedback for future users. In addition, comments can only be deleted by the user who created them. The Bug Search Module also

supports performance by helping all users locate bugs quickly in the system without having to manually scan through every record.

`SELECT * FROM `admin``

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	admin_id	name	email	password
☐ Edit Copy Delete	3	Admin One	admin1@test.com	\$2y\$10\$u9H1.oDjL.kHaNYuL21JS.3gCOJXG5FsBwl6zU9caly...
☐ Edit Copy Delete	4	Admin Two	admin2@test.com	\$2y\$10\$u9H1.oDjL.kHaNYuL21JS.3gCOJXG5FsBwl6zU9caly...

↑ ☐ Check all With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

```
<?php
session_start();
include 'db.php';

if (!isset($_SESSION['role']) || $_SESSION['role'] != "admin") {
    header("Location: index.php");
    exit();
}
```

Interface Requirements

The system is a web-based application developed using PHP, HTML, and MySQL. PHP and HTML were used to build the web pages and handle the system's functionality, while MySQL was used as the back-end database to store role accounts, bugs and all their details, comments, and ratings.

- Application Type: Web-based
- Front-end Software: PHP and HTML
- Back-end Software: MySQL
- Operating System: Windows and Linux
- Supported Browsers: Google Chrome, Internet Explorer

- Editor and Tools Used: cPanel File Manager and phpMyAdmin(mysql)

The interface uses a consistent page structure across the bug tracker system. Each role has its own dashboard with a menu that organizes the pages available to that role. For example, Users have access to pages such as My Bugs, Submit Bug, Search Bugs, and they also have a rate option on their resolved bugs. Admins menu is organized by Bug Management and Staff Management with access to pages such as Assign Bug, Approve Solutions, and Edit Bug. Staff Members have a dashboard that organizes assigned bugs into sections such as New Assignments, In Progress, and Resolved.

The System also uses forms and section cards through the interface. Forms are used for actions such as logging in, submitting bugs, adding staff members, editing bug details, submitting solutions, and adding comments. Section cards are used to organize bug records, staff records, search results, and status information in clear visual layout. This creates a uniform look and feel between pages while keeping the interface connected to the system's main functions and database.

Testing

Testing was an important part of our Bug Tracker system because we needed to make sure each role and feature worked correctly. We tested the system by using different accounts for Admin, Staff, and User. First, we tested the login function. Each role was tested with a correct email and password to make sure the user was redirected to the correct dashboard. Admin users were redirected to the Admin Dashboard, Staff users were redirected to the Staff Dashboard, and regular users were redirected to the User Dashboard. We also tested invalid login information to make sure the system did not allow unauthorized access.

Second, we tested role-based access control. Each page checks the user's session role before loading. For example, a staff member should not be able to access the Admin Dashboard. If the role does not match, the system redirects the user back to the login page. This helped us confirm that each role could only access the features assigned to them.

Third, we tested the bug workflow. A user submitted a bug with required information such as title, severity, impact, and source application. Then the admin assigned the bug to a staff member. The staff member accepted the bug and submitted a solution. After that, the admin reviewed and approved the solution, and the bug status was updated to resolved.

Fourth, we tested comments and ratings. Users and staff were able to submit comments connected to a bug. We also tested that users could rate a resolved bug using the 1-to-5 rating system.

Finally, we tested the database connection and data storage. We checked that submitted bugs, comments, ratings, and status updates were saved correctly in the MySQL database. This helped confirm that the front-end pages and back-end database were working together.

Roles of Team Members

Our team worked collaboratively throughout the project, with each member contributing to multiple parts. All team members worked together to create the project presentation slides and final project report.

Mengjia Yu was responsible for designing and developing the role-based home pages for the Bug Tracker system. He worked on the Admin, Staff, and User dashboard pages and helped make sure each role only sees the features related to their permissions. He also contributed to the CSS styling of the website, including the layout, menu design, buttons, cards, and overall visual consistency. In addition, he helped test the login redirection process to confirm that Admin, Staff, and User accounts were directed to the correct dashboard.

Roxanne Caicedo worked on the login, register, and logout pages. She designed the logo and color scheme of the website. She also formatted the dashboards to display the logo consistently, added secondary headers for each role, and created a consistent menu design with role-specific menu options. In addition, she worked on the forms used throughout the system for submitting a bug, assigning a bug, resolving a bug, as well as the section cards displayed on each dashboard home page and in search results. She implemented the bug data flow from creation to resolution using the status field. Finally, she helped test this data flow through integration testing to ensure that a bug was correctly passed from role to role with each key action.

Jorge Cuautle worked on the Non-Functional Requirements and Testing of the project. Designing test cases such as Usability, Security and workflow were one of my main tasks. Using Integration and Unit Testing I was able to see modules working properly as they were intended to. Security was another thing Jorge worked on as he needed pages to be secured for users to

be blocked off on certain vulnerable pages. Along with that I was tasked to hash the passwords for enhanced security for all users.

Taiwo Ojo was responsible for designing and implementing the database for the Bug Tracker system. He used SQL to create and manage the database structure, including tables for users, bugs, roles, and status tracking. He ensured proper relationships between tables using primary and foreign keys to maintain data integrity. Taiwo also handled data storage for bug submissions, assignments, and updates throughout the bug lifecycle. In addition, he worked on writing queries to retrieve and update data efficiently, supporting key system features such as role-based dashboards and bug status tracking.

Outcomes of the Project

By the end of this project, we successfully developed a working bug tracker system with the main features we originally planned. The system includes user authentication, role-based dashboards, bug reporting, assignment, resolution, and tracking. The project also resulted in clear role-based dashboards for Admin, Staff, and User accounts. This helped separate responsibilities within the system and made the workflow easier to understand. The interface styling improved the overall usability of the application by making menus, forms, and dashboard sections more organized and consistent.

Limitations

Although our bug tracker system is functional, there are still some limitations. First, the system has limited security features. While passwords are stored securely, more advanced protections such as multi-factor authentication are not implemented. With more time we would have also included email authentication where registration is completed through a link sent via email to log-in for the first time. Second, the search function is basic and only supports simple keyword matching. It does not support advanced filters such as searching by date, severity, or assigned staff. Third, the user interface is simple and could be improved with more interactive features and better responsiveness on mobile devices or a mobile app design and implementation. In addition, the system currently does not support file uploads for bug reports, such as screenshots or attachments, which could help users provide more detailed information.

Finally, due to time constraints, some features such as notifications and real-time updates were not fully implemented.

Conclusion

In conclusion, our bug tracker system successfully meets the main objectives of the project. We developed a web-based application that allows users to submit bugs, staff members to resolve them, and administrators to manage the entire workflow. The system includes important features such as login authentication, role-based dashboards, bug tracking, comments, and ratings. Throughout the project, we improved our skills in PHP, MySQL, HTML, and CSS. We also gained experience in teamwork, problem-solving, and system design. Although there are some limitations, the system provides a solid foundation that can be further improved in the future. Overall, this project helped us better understand how real-world web applications are designed and implemented.